Greenhouse CO_2 Controller

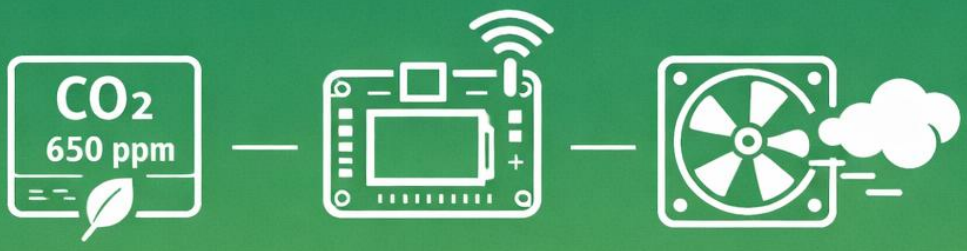

Table of Contents

Greenhouse CO2 Controller 4

User Manual and Program Documentation 4

1. Introduction 4

2. System Overview 4

2.1 Main Functions 5

2.2 Hardware Used 6

3. Local User Interface 6

4. Changing CO2 Target Locally 7

5. Changing Wi-Fi Profile 8

6. Remote Monitoring 8

7. Remote Setpoint Control 9

8. Status Indicators and Error Handling 10

8.1 Wi-Fi Status 10

8.2 Cloud Status 11

8.3 Modbus Status 11

8.4 EEPROM Status 11

8.5 Error Log 11

9. Troubleshooting 11

10. Software Architecture 12

10.1 input_task 13

10.2 display_task 13

10.3 eeprom_task 13

10.4 modbus_task 13

10.5 pressure_task 13

10.6 controller_task 14

10.7 wifi_task 14

10.8 cloud_task 14

10.9 Shared State and Synchronization 14

11. Control Logic 15

11.1 CO2 Setpoint and Hysteresis 15

11.2 Fan Control 15

11.3 CO2 Valve Control 15

11.4 Safety Mode 15

11.5 Local and Remote Setpoint Consistency 16

11.6 Pressure Measurement Usage 16

12. Persistent Configuration 16
13. Communication Interfaces 16
13.1 Modbus 17
13.2 I2C 17
13.3 GPIO 18
13.4 Wi-Fi 18
13.5 HTTPS/TLS Cloud Communication 18
13.6 Browser Dashboard Communication 18
14. Cloud Integration 18
14.1 Telemetry Upload 19
14.2 ThingSpeak Channel Usage 19
14.3 Remote Commands with TalkBack 20
14.4 Local Dashboard as Cloud Client 20
14.5 Benefits of the Final Cloud Architecture 20
15. User Interface Design 21
15.1 Page Structure 21
15.2 Navigation Logic 21
15.3 Local Editing Philosophy 21
15.4 Status Visibility 21
16. Pin Mapping and Hardware Connections 22
16.1 Rotary Encoder 22
16.2 CO2 Valve Control 22
16.3 Modbus UART 22
16.4 I2C Buses 23
16.5 OLED and EEPROM Addresses 23
16.6 Wireless Connectivity 23
17. Project File Structure 23
17.1 Embedded Firmware 23
17.2 Browser Dashboard 24
17.3 System-Level Structure 24
18. Limitations and Future Improvements 24
18.1 Current Limitations 24
18.2 Possible Future Improvements 25
18.3 Design Trade-Offs 25
19. Conclusion 25

Greenhouse CO2 Controller

User Manual and Program Documentation

1. Introduction

The Greenhouse CO2 Controller is an embedded control system built around the Raspberry Pi Pico W and FreeRTOS. The purpose of the system is to monitor greenhouse conditions and maintain the desired carbon dioxide level by controlling a fan and a CO2 valve. In addition to local control, the system supports Wi-Fi connectivity, cloud telemetry, remote setpoint control, and a browser-based dashboard.

The project combines real-time sensing, actuator control, persistent configuration storage, and remote communication in one integrated system. A local OLED display and rotary encoder provide a simple user interface directly on the device, while ThingSpeak and a local web dashboard allow remote monitoring and control.

The system reads environmental data from multiple sources. CO2 concentration, temperature, humidity, and fan pulse information are received through Modbus communication with the simulator or external device. Differential pressure is measured separately using an SDP610 pressure sensor connected through I2C. Based on these inputs, the controller adjusts fan operation and controls the CO2 valve according to the configured target and safety rules.

This document contains both the user manual and the technical program documentation for the final implementation of the Greenhouse CO2 Controller.

2. System Overview

The Greenhouse CO2 Controller is designed as a multitask embedded system running on FreeRTOS. The system continuously reads sensor data, updates the local user interface, stores persistent settings in EEPROM, manages Wi-Fi connectivity, sends telemetry to the cloud, and receives remote setpoint commands.

The controller can be operated locally through the OLED interface and rotary encoder. The user can browse status pages, change the CO2 target, and select one of the available Wi-Fi profiles directly from the device. Saved settings are stored in EEPROM and restored after power cycling.

For remote operation, the system sends telemetry to ThingSpeak using HTTPS/TLS. A browser-based dashboard reads the latest values and history data from ThingSpeak and allows the user to send a new CO2 target remotely. Remote target commands are delivered to the Pico W through ThingSpeak TalkBack.

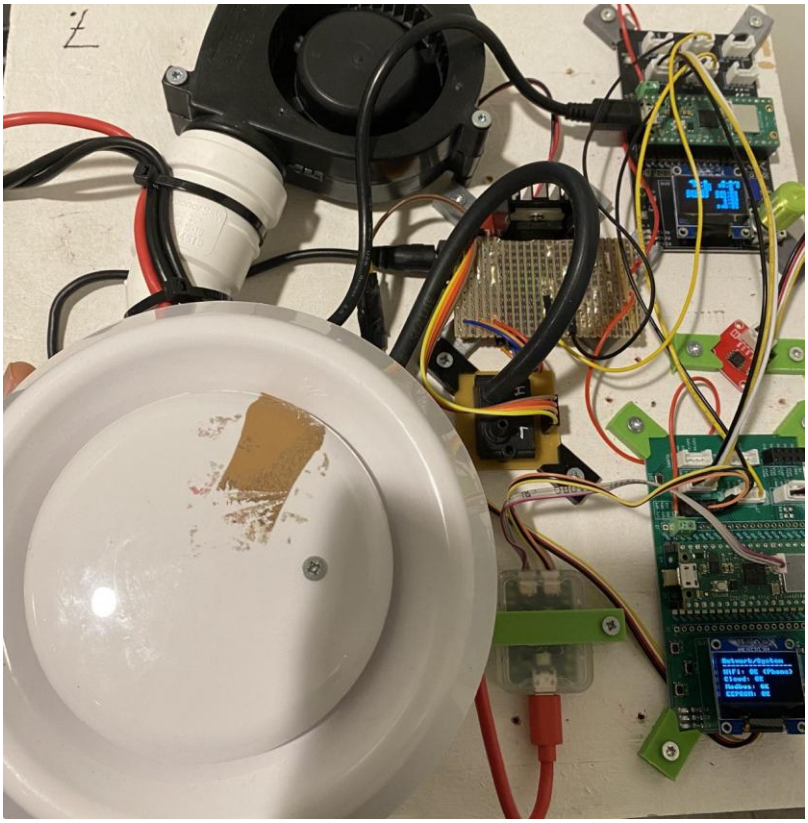

Figure 1. Overall view of the Greenhouse CO2 Controller prototype.

2.1 Main Functions

The main functions of the system are:

- measurement of CO2 concentration
- measurement of temperature and relative humidity
- measurement of differential pressure
- fan control based on CO2 level and controller logic
- CO2 valve control with pulse and stabilization timing
- local user interface on OLED display
- persistent storage of selected settings in EEPROM
- Wi-Fi profile selection from the device UI
- cloud telemetry upload to ThingSpeak
- remote CO2 target control through ThingSpeak TalkBack
- browser-based remote monitoring dashboard

2.2 Hardware Used

The final system uses the following main hardware components:

- Raspberry Pi Pico W as the main controller
- OLED display for local user interface
- rotary encoder with push button for user input
- EEPROM for persistent configuration storage
- Modbus-connected simulator or Produal/MIO device for CO2, temperature, humidity, and fan pulse values
- SDP610 differential pressure sensor connected through I2C
- fan control output
- CO2 valve control output
- Wi-Fi connection through the Pico W wireless interface

3. Local User Interface

The local user interface is implemented using a small OLED display and a rotary encoder with a push button. The display presents the current system status and allows the user to access different pages. The rotary encoder is used both for page navigation and for editing selected values.

In normal mode, rotating the encoder moves between pages. Pressing the encoder button on an editable page enters edit mode. In edit mode, rotating the encoder changes the selected value or option, and pressing the button again saves the new value.

The local interface includes pages for the main sensor overview, detailed sensor values, CO2 target editing, Wi-Fi profile selection, network and system status, and error information.

Figure 2. Local OLED interface showing the main status page.

Figure 3. Local OLED interface showing system and network status.

Figure 4. Local OLED interface for Wi-Fi profile selection.

4. Changing CO2 Target Locally

The CO2 target can be adjusted directly from the local OLED user interface. This allows the user to configure the controller without using the browser dashboard or cloud connection.

To change the CO2 target, first rotate the encoder until the **Set CO2 Target** page is displayed. Press the button9 once to enter edit mode. In edit mode, rotating the encoder changes the target value in fixed steps. When the desired value is shown on the display, press the button9 again to save the new target.

After saving, the value is written to EEPROM. This means the selected CO2 target is preserved even if the device is powered off or restarted. The saved setpoint is also used by the control logic and is included in the telemetry sent to ThingSpeak.

If a remote target is later received through ThingSpeak TalkBack, the system applies the new valid setpoint automatically. In this way, the controller supports both local and remote target updates.

Local CO2 Target Change Procedure

1. Rotate the encoder until the **Set CO2 Target** page appears.
2. Press the button9 to enter edit mode.
3. Rotate the encoder to increase or decrease the target value.
4. Press the button9 again to save the value.
5. The value is stored in EEPROM and remains available after restart.

5. Changing Wi-Fi Profile

The system supports local Wi-Fi profile selection directly from the OLED user interface. This allows the user to choose between predefined network profiles without modifying the firmware.

The current implementation contains two predefined Wi-Fi profiles:

- a phone hotspot profile
- a school Wi-Fi profile

To select a Wi-Fi profile, rotate the encoder until the **WiFi Select** page is shown. Press the button9 once to enter edit mode. In edit mode, rotate the encoder to switch between the available profiles. Press the button again to save the selected profile.

The selected Wi-Fi profile is stored in EEPROM together with the other persistent configuration values. After saving, the system reconnects using the selected profile. This allows the device to switch networks without manual reflashing or code changes.

The currently selected profile is also shown in the system status information, so the user can verify which network profile is active.

Wi-Fi Profile Change Procedure

1. Rotate the encoder until the **WiFi Select** page appears.
2. Press the button9 to enter edit mode.
3. Rotate the encoder to select the desired profile.
4. Press the button9 again to save the selected profile.
5. The profile is stored in EEPROM.
6. The device reconnects using the selected Wi-Fi profile.

Notes

- Only predefined Wi-Fi profiles are supported in the final implementation.
- Manual text entry of SSID and password is not implemented.
- This approach was chosen because it is practical and reliable for a small OLED-and-encoder interface.
- The feature still provides local network selection and persistent storage of the selected profile.

6. Remote Monitoring

The Greenhouse CO2 Controller supports remote monitoring through ThingSpeak and a browser-based dashboard. Sensor values and controller information are uploaded periodically from the Pico W to the cloud. The dashboard reads this data and presents it in a user-friendly format.

The system uploads the following values to ThingSpeak:

- **field1** = CO2 concentration

- **field2** = relative humidity
- **field3** = temperature
- **field4** = fan speed
- **field5** = CO2 setpoint

This telemetry allows the user to monitor the current greenhouse state remotely and review recent trends through charts.

In addition to ThingSpeak, a local Node/Express dashboard is used as a browser interface. The dashboard reads the latest values and history from ThingSpeak and displays them in real time. It also provides a convenient way to submit a new remote CO2 target.

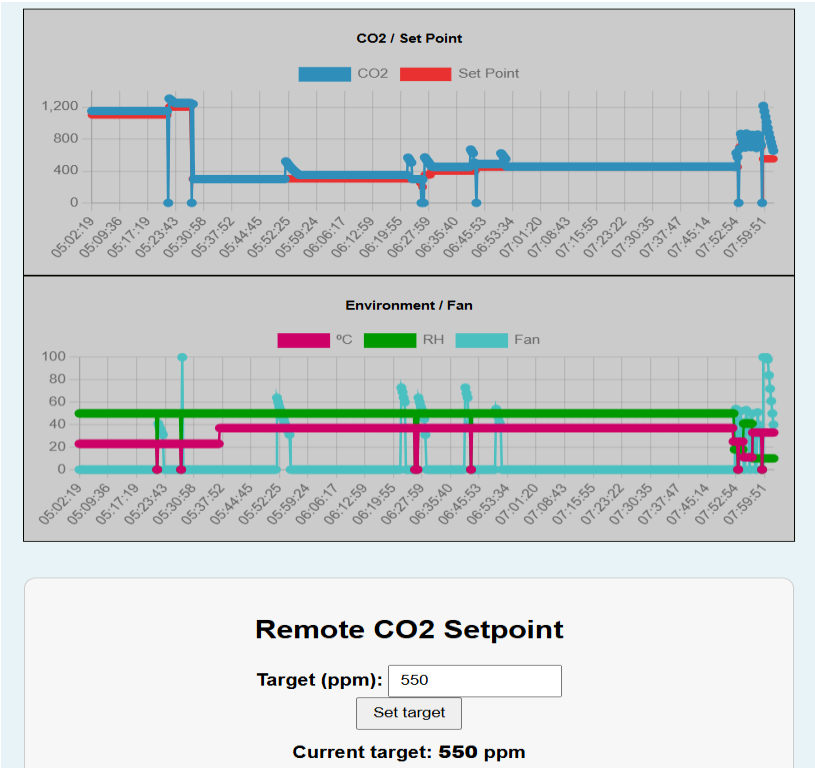

Figure 5. Browser-based dashboard reading live and historical data from ThingSpeak.

7. Remote Setpoint Control

In addition to local target editing, the system supports remote CO2 target control through the browser dashboard and ThingSpeak TalkBack. This makes it possible to change the target without touching the device physically.

When the user enters a new target value in the dashboard, the server sends the command to ThingSpeak TalkBack. The Pico W periodically polls TalkBack for pending commands. If a valid setpoint command is found, the controller applies the new target automatically.

The accepted command format is a numeric setpoint value or a command in the form SP=900. When a valid command is received, the new target is applied locally, stored in EEPROM, and later visible in both local UI and cloud telemetry.

This mechanism keeps local and remote control synchronized. After a remote setpoint has been applied, the updated value appears on the OLED display, in the browser dashboard, and in the telemetry field used for the setpoint.

Remote Setpoint Change Procedure

1. Open the browser dashboard.
2. Enter the desired CO2 target value in the input field.
3. Press the **Set target** button.
4. The dashboard sends the command to ThingSpeak TalkBack.
5. The Pico W reads the command during its next polling cycle.
6. If the value is valid, the new setpoint is applied and stored in EEPROM.
7. The updated value becomes visible locally and remotely.

Notes

- Only valid setpoint values within the accepted range are applied.
- Invalid commands are ignored.
- Remote commands do not require reflashing or restarting the device.
- The final setpoint visible on the display and in telemetry always reflects the current active controller target.

---

8. Status Indicators and Error Handling

The system provides status information through the OLED display and internal error messages. This makes it easier for the user to understand whether the system is operating normally or whether there is a communication or hardware problem.

The **Network/System** page shows the status of the main system interfaces and services. These indicators help the user confirm that the device is connected and functioning correctly.

8.1 Wi-Fi Status

The Wi-Fi status indicates whether the Pico W is currently connected to the selected wireless network.

- **OK** means that the device is connected successfully.
- **DOWN** means that the device is not currently connected.

If Wi-Fi is down, cloud communication will also fail until connectivity is restored.

8.2 Cloud Status

The cloud status indicates whether telemetry upload or remote cloud communication is functioning correctly.

- **OK** means that the device is able to communicate with ThingSpeak.
- **DOWN** means that telemetry or cloud communication has failed.

A cloud failure does not stop local control logic, but remote monitoring and remote setpoint control may become unavailable.

8.3 Modbus Status

The Modbus status indicates the communication state of the Modbus-connected data source.

- **OK** means that CO2, temperature, humidity, and fan pulse data are being read successfully.
- **ERROR** means that the Modbus communication failed or returned invalid data.

If Modbus communication fails, the displayed sensor values may stop updating and control performance may be affected.

8.4 EEPROM Status

The EEPROM status indicates whether reading and writing persistent configuration data is working properly.

- **OK** means that configuration can be stored and restored.
- **ERROR** means that EEPROM access failed.

If EEPROM is not working correctly, saved values such as setpoint or Wi-Fi profile may not persist after restart.

8.5 Error Log

The **Error Log** page shows the latest stored error or system message. This is useful for identifying connection failures, cloud errors, or other operational issues.

Typical messages may include:

- Wi-Fi connection error codes
- cloud communication failures
- configuration save messages
- reconnect notifications

9. Troubleshooting

This section provides basic troubleshooting instructions for common situations.

Wi-Fi is DOWN

- Check that the selected Wi-Fi profile is correct.

- Verify that the target network is available.
- If using a phone hotspot, confirm that the hotspot is active and within range.
- If needed, select the Wi-Fi profile again from the OLED interface.

Cloud is DOWN

- Check that Wi-Fi is connected.
- Verify that internet access is available.
- Confirm that ThingSpeak credentials are correct.
- Check whether ThingSpeak is reachable from the current network.

Modbus shows ERROR

- Check wiring between the controller and the simulator/device.
- Verify that the external device is powered and responding.
- Confirm that the Modbus configuration matches the expected settings.

Setpoint does not update remotely

- Verify that the browser dashboard submitted the command successfully.
- Confirm that the command was added to ThingSpeak TalkBack.
- Wait for the next TalkBack polling cycle.
- Check the Error Log page for cloud communication messages.

Saved values are not restored

- Check EEPROM status on the display.
- Verify that the value was saved correctly before power-off.
- If needed, save the value again and restart the device.

10. Software Architecture

The Greenhouse CO2 Controller is implemented as a multitask embedded application running on FreeRTOS. The software is divided into separate tasks, each responsible for one main subsystem. This structure improves clarity, keeps the logic modular, and makes the system easier to test and maintain.

The tasks communicate through shared state protected by synchronization primitives. Sensor values, controller states, configuration values, and system status flags are stored in a shared application state structure. Access to this state is protected to avoid race conditions between tasks.

The final implementation includes the following main tasks.

10.1 input_task

The input_task handles the rotary encoder and push button. Its main responsibilities are page navigation, edit-mode handling, local setpoint adjustment, and Wi-Fi profile selection.

When the user is in normal mode, encoder rotation changes the current page. When the user is on an editable page and presses the button, the task enters edit mode. In edit mode, encoder rotation changes the current setting and a second button press saves the value.

This task is responsible for setting the save request flags that later cause the EEPROM task to store updated values.

10.2 display_task

The display_task updates the OLED display. It reads the current shared state and renders the appropriate page contents, including live measurements, target values, network status, and error messages.

The display is page-based and includes dedicated views for the main overview, sensor details, target editing, Wi-Fi selection, system status, and error information.

10.3 eeprom_task

The eeprom_task is responsible for persistent configuration handling. At startup, it reads the stored configuration from EEPROM and restores the last saved values. During operation, it watches for save requests triggered by the user interface and writes the updated configuration back to EEPROM.

In the final implementation, EEPROM stores at least:

- the CO2 setpoint
- the selected Wi-Fi profile

This ensures that important settings remain available after power cycling.

10.4 modbus_task

The modbus_task handles Modbus communication with the simulator or external device. It reads the values required by the control system, including:

- CO2 concentration
- temperature
- relative humidity
- fan pulse information

The task also writes the fan command value back to the external device when needed.

10.5 pressure_task

The pressure_task reads the SDP610 differential pressure sensor through I2C. The measured pressure value is converted into the internal representation used by the application state and displayed in the local interface.

This task is separated from the Modbus logic because the pressure sensor uses a different hardware interface and data source.

10.6 controller_task

The controller_task implements the main greenhouse control logic. It continuously evaluates the measured CO_2 concentration relative to the configured target and determines:

- whether the fan should be off or running
- what fan command level should be used
- whether the CO_2 valve should be opened
- whether safety mode must be activated

The controller task is central to the behavior of the system and translates measured data into control actions.

10.7 wifi_task

The wifi_task manages the Pico W wireless connection. It uses the selected Wi-Fi profile from the shared state and attempts to connect to the corresponding network. If the selected profile changes, the task reconnects using the new profile.

This task updates Wi-Fi-related status flags and allows the network to be changed locally from the OLED interface.

10.8 cloud_task

The cloud_task is responsible for cloud communication. It sends telemetry to ThingSpeak using HTTPS/TLS and periodically polls ThingSpeak TalkBack for remote setpoint commands.

This task also updates cloud communication status information and keeps the system synchronized with remote control requests.

10.9 Shared State and Synchronization

The application uses a shared global state structure to store:

- sensor values
- setpoint values
- control outputs
- page state
- edit state
- Wi-Fi selection
- status flags
- error strings

Because multiple tasks read and write this state, synchronization is required. Mutex-based protection is used to ensure safe access and to prevent inconsistent updates.

11. Control Logic

The control logic of the Greenhouse CO2 Controller is based on the measured CO2 concentration, the configured setpoint, and a set of safety and timing rules. The purpose of the controller is to maintain the desired CO2 level while avoiding unstable behavior and handling fault situations safely.

11.1 CO2 Setpoint and Hysteresis

The controller compares the current CO2 value to the active target setpoint. A hysteresis margin is used so that the system does not switch outputs too frequently when the measured CO2 value is close to the target.

When the CO2 value rises above the target plus hysteresis, the fan can be enabled or increased. When the CO2 value drops below the target, the controller may reduce or disable fan operation depending on the current state and control logic.

This hysteresis-based behavior helps reduce unnecessary switching and improves stability.

11.2 Fan Control

The fan is controlled through a command value written to the Modbus-connected device. The final fan command depends on the current CO2 level and the controller rules.

The fan state is interpreted using both the requested command and the measured pulse feedback:

- **OFF** means the fan is not requested
- **RUN** means the fan is requested and pulses are detected
- **FAULT / STOPPED** means the fan is requested but no pulses are detected

This allows the system to distinguish between a fan that is intentionally off and a fan that should be running but is not responding correctly.

11.3 CO2 Valve Control

The CO2 valve is controlled through a dedicated GPIO output. The final implementation includes pulse and stabilization logic:

- the valve may open for a limited pulse duration
- after the pulse, a stabilization delay is applied
- the controller waits before opening the valve again

This prevents continuous uncontrolled valve operation and gives the system time to observe the effect of gas injection before sending another valve pulse.

11.4 Safety Mode

The controller includes a safety mode that is activated when the CO2 concentration reaches a critical limit. In the final implementation, this happens when CO2 reaches or exceeds the defined safety threshold.

When safety mode is active, the system prioritizes safe operation over normal target tracking. This protects the environment from excessive CO2 concentration and ensures that the system reacts appropriately in abnormal conditions.

11.5 Local and Remote Setpoint Consistency

The controller supports both local and remote setpoint changes. Local changes are made through the OLED interface, while remote changes arrive through ThingSpeak TalkBack.

When a new valid setpoint is received remotely, the controller applies it, stores it in EEPROM, and uses it as the active control target. The same value is then shown in the local interface and included in cloud telemetry. This keeps the local and remote system state consistent.

11.6 Pressure Measurement Usage

Differential pressure is measured and displayed as part of the system state. In the final version of the project, this value is primarily used for monitoring and display rather than as the main input to the CO2 control algorithm.

However, it still provides useful diagnostic information and extends the monitoring capabilities of the system.

12. Persistent Configuration

The system stores important user settings in EEPROM so that configuration is preserved after restart or power loss. This improves usability and makes the device suitable for repeated practical use without requiring the user to re-enter values after every power cycle.

The persistent configuration includes:

- the CO2 target setpoint
- the selected Wi-Fi profile

At startup, the EEPROM task reads the saved configuration and restores the values into the shared application state. During operation, when the user saves a new target or Wi-Fi profile, the EEPROM task writes the updated configuration back to memory.

To improve reliability, the stored configuration includes versioning and integrity checking. This allows the firmware to validate the saved data and helps support future changes in configuration structure.

13. Communication Interfaces

The Greenhouse CO2 Controller uses several communication interfaces to connect sensors, storage, user interface hardware, and cloud services. Each interface is assigned a clear purpose in the system architecture.

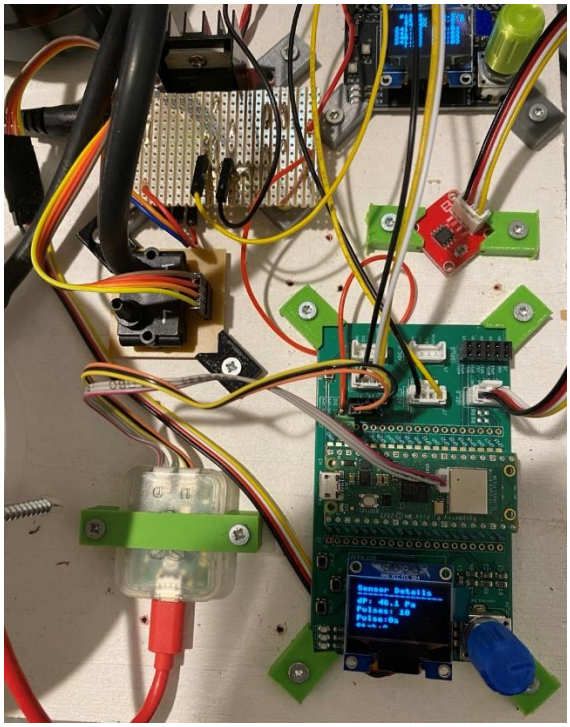

Figure 6. Main controller board with Pico W, OLED display, rotary encoder, EEPROM, and pressure sensor connections.

13.1 Modbus

Modbus communication is used for exchanging data with the simulator or external device. Through this interface, the controller reads the main environmental values required for greenhouse operation:

- CO2 concentration
- temperature
- relative humidity
- fan pulse information

The controller also writes the fan command back through Modbus. This makes Modbus the main interface between the embedded controller and the external process or simulator.

13.2 I2C

I2C is used for peripheral devices connected directly to the Pico. In the final implementation, I2C is used for:

- the OLED display
- the EEPROM
- the SDP610 differential pressure sensor

The project uses separate I2C buses according to the hardware design. This keeps communication organized and avoids unnecessary conflicts between peripherals.

13.3 GPIO

General-purpose input/output pins are used for direct hardware control and user input. GPIO is used for:

- the rotary encoder channels
- the rotary encoder push button
- the CO2 valve control signal
- other direct digital signals used by the controller

GPIO outputs are used where low-level direct control is required, while GPIO inputs are used for user interaction and signal monitoring.

13.4 Wi-Fi

The Pico W provides the wireless network connection used for cloud communication. The selected Wi-Fi profile determines which network credentials are used. In the final implementation, Wi-Fi can be selected from predefined profiles through the local OLED interface.

The Wi-Fi task maintains connectivity and reconnects if the selected profile is changed or if the connection is lost.

13.5 HTTPS/TLS Cloud Communication

Telemetry upload and remote command polling are performed over HTTPS/TLS. This is used for communication with ThingSpeak and ThingSpeak TalkBack.

Using TLS allows the system to communicate with the cloud service securely. The cloud task creates outgoing HTTPS requests for:

- telemetry upload
- TalkBack command polling

13.6 Browser Dashboard Communication

The local browser dashboard is implemented with Node/Express and WebSocket support. The dashboard reads current and historical data from ThingSpeak and pushes live updates to the browser.

This creates a layered communication model:

- Pico W communicates with ThingSpeak
- the local dashboard server reads data from ThingSpeak
- the browser receives live dashboard updates through WebSocket

14. Cloud Integration

Cloud integration is implemented using ThingSpeak for telemetry and ThingSpeak TalkBack for remote commands. This makes it possible to monitor the system remotely and update the CO2 target without direct physical access to the controller.

Figure 7. ThingSpeak channel used for telemetry visualization.

14.1 Telemetry Upload

The Pico W periodically sends telemetry data to ThingSpeak. The data is formatted according to the project requirements using the following fields:

- **field1** = CO2 concentration
- **field2** = relative humidity
- **field3** = temperature
- **field4** = fan speed
- **field5** = CO2 setpoint

Telemetry is sent over HTTPS/TLS from the cloud task. The final implementation uses ThingSpeak as the primary cloud backend instead of the earlier local server-only approach.

14.2 ThingSpeak Channel Usage

A dedicated ThingSpeak channel is used to store and visualize the greenhouse data. The cloud charts provide a simple way to verify that telemetry is arriving correctly and to review recent system behavior over time.

Because the channel fields match the project specification, the cloud representation is directly aligned with the system's internal telemetry model.

14.3 Remote Commands with TalkBack

Remote setpoint control is implemented using ThingSpeak TalkBack. The browser dashboard sends a command to the TalkBack queue when the user requests a new CO2 target.

The Pico W polls TalkBack periodically. When a valid command is found, the firmware:

1. parses the received command
2. validates the requested setpoint
3. applies the new value
4. stores it in EEPROM
5. continues reporting the updated value through telemetry

This creates a complete local–remote synchronization loop.

14.4 Local Dashboard as Cloud Client

In the final architecture, the local browser dashboard no longer acts as the primary telemetry backend. Instead, it reads current values and history data from ThingSpeak.

This means the dashboard behaves as a cloud client:

- it requests the latest entry from ThingSpeak
- it requests recent historical entries for chart display
- it submits new target commands through TalkBack
- it forwards live updates to the browser through WebSocket

This design keeps the browser interface available while using ThingSpeak as the central data source.

14.5 Benefits of the Final Cloud Architecture

The final ThingSpeak-based solution provides several practical advantages:

- no permanent local telemetry server is required for the Pico
- telemetry format matches the project specification
- cloud charts are available immediately
- remote setpoint control is integrated through TalkBack
- the browser dashboard can still be used as a familiar interface

This architecture is more robust and better aligned with the project requirements than a local-only cloud simulation.

15. User Interface Design

The local user interface is designed around a small OLED screen and a single rotary encoder with push button9. Because the available display space and input method are limited, the interface follows a simple page-based structure.

The main design principles are:

- clear page separation
- minimal required button presses
- consistent edit/save behavior
- readable status presentation

15.1 Page Structure

The OLED interface is organized into multiple pages, each with a specific function:

- Home
- Sensor Details
- Set CO2 Target
- WiFi Select
- Network/System
- Error Log

This keeps each screen focused and reduces clutter.

15.2 Navigation Logic

In normal mode, rotating the encoder changes the active page. On editable pages, pressing the button9 enters edit mode. In edit mode, rotating the encoder modifies the current value or option, and pressing again saves the selection.

This interaction model is used consistently across the adjustable settings pages.

15.3 Local Editing Philosophy

The interface is designed to avoid complicated text input on the OLED. For this reason:

- the CO2 target is edited as a numeric value
- Wi-Fi is selected from predefined profiles rather than entered manually

This approach is practical for the available hardware and keeps the local UI reliable and easy to use.

15.4 Status Visibility

The interface is also designed to make system status immediately visible. Important information such as Wi-Fi connectivity, cloud status, Modbus status, EEPROM status, selected Wi-Fi profile, and recent errors can be viewed directly on the device.

This helps with both normal operation and troubleshooting.

16. Pin Mapping and Hardware Connections

The final implementation uses the following main pin assignments and hardware interfaces.

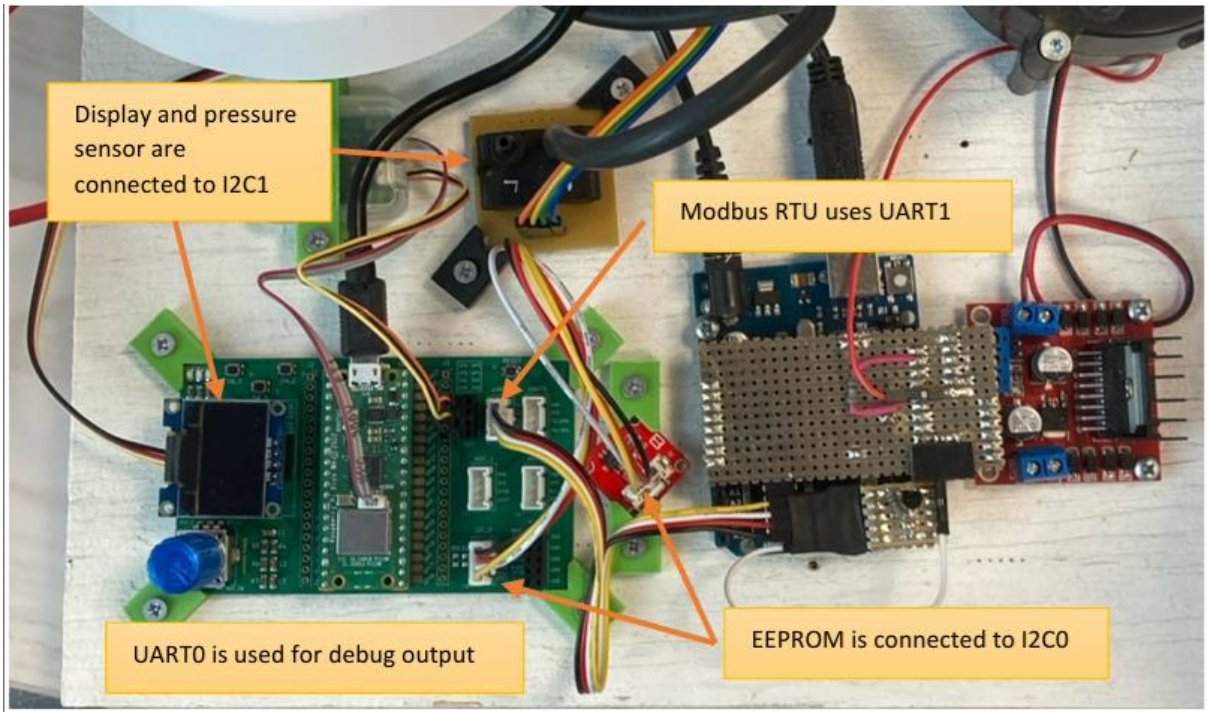

Figure 8. Hardware connection overview of the prototype system.

16.1 Rotary Encoder

The rotary encoder is used for page navigation, editing, and saving values through the local OLED interface.

- **ROT_A_PIN = GP10**
- **ROT_B_PIN = GP11**
- **BUTTON9_PIN = GP9**

The two encoder channels are read as digital inputs, and the push button is used to enter and confirm edit mode.

16.2 CO2 Valve Control

The CO2 valve is controlled through a digital GPIO output.

- **CO2_RELAY_PIN = GP27**

This output is used by the controller task to activate the CO2 valve according to the pulse and stabilization logic.

16.3 Modbus UART

Modbus communication is handled through UART.

- **MODBUS UART = UART1**

- **MODBUS_TX_PIN = GP4**
- **MODBUS_RX_PIN = GP5**
- **MODBUS_BAUD = 9600**

This interface is used for communication with the simulator or external device that provides CO2, temperature, humidity, and fan pulse data.

16.4 I2C Buses

Two I2C buses are used in the project.

I2C0

- **SDA = GP16**
- **SCL = GP17**

I2C0 is used for EEPROM communication.

I2C1

- **SDA = GP14**
- **SCL = GP15**

I2C1 is used for the OLED display and the SDP610 pressure sensor.

16.5 OLED and EEPROM Addresses

The project uses the following device addresses:

- **OLED primary address = 0x3C**
- **OLED secondary address = 0x3D**
- **EEPROM address = 0x50**

The OLED detection logic checks the supported display addresses automatically.

16.6 Wireless Connectivity

Wireless communication is provided by the integrated Wi-Fi hardware of the Raspberry Pi Pico W. The firmware uses predefined Wi-Fi profiles and selects the active profile through the local user interface.

17. Project File Structure

The project consists of embedded firmware and browser dashboard components.

17.1 Embedded Firmware

The embedded side is built around the FreeRTOS-based firmware.

Important firmware modules include:

- **main.cpp**
  Main application logic, task creation, shared state, local UI handling, controller logic, Wi-Fi management, cloud communication, and EEPROM integration.

- **I2C-related source files**
  Used for OLED, EEPROM, and pressure sensor communication.

- **UART / Modbus source files**
  Used for reading environmental values and controlling the fan through Modbus.

- **TLS-related source files**
  Used for secure HTTPS/TLS communication with ThingSpeak and ThingSpeak TalkBack.

17.2 Browser Dashboard

The browser dashboard is implemented using Node/Express and client-side JavaScript.

Main dashboard files include:

- **index.js**
  Backend server logic for reading data from ThingSpeak, forwarding live updates through WebSocket, and sending remote setpoint commands to TalkBack.

- **home.js**
  Frontend dashboard logic for charts, live value updates, and remote target submission.

- **home.ejs**
  Main browser dashboard page template.

17.3 System-Level Structure

From a functional point of view, the project can be divided into three layers:

1. **Embedded control layer**
   Handles local sensing, control, persistence, display, and Wi-Fi.

2. **Cloud layer**
   ThingSpeak stores telemetry and TalkBack carries remote commands.

3. **Dashboard layer**
   The local browser server reads cloud data and presents it to the user.

This layered structure makes the system easier to explain, debug, and demonstrate.

18. Limitations and Future Improvements

The final version of the project is fully functional and demonstrates both local and remote operation. However, some limitations remain and could be improved in future development.

18.1 Current Limitations

- The local Wi-Fi configuration supports only predefined profiles.
  Manual entry of SSID and password through the OLED interface is not implemented.

- The school Wi-Fi profile could not be fully tested in advance outside the final environment. The profile was included based on the available credentials and intended use case.
- The local browser dashboard depends on ThingSpeak as the source of truth. It does not operate as a fully independent telemetry backend anymore.
- Differential pressure is currently used mainly for monitoring and display, not as a core control variable in the final control algorithm.

18.2 Possible Future Improvements

Possible future extensions include:

- manual text-based network parameter entry from the local UI
- additional cloud commands beyond the CO2 setpoint
- longer-term logging and event history
- more advanced browser dashboard views
- alarm handling and notification features
- more advanced fan control or adaptive control strategies
- broader use of pressure data in the control algorithm

18.3 Design Trade-Offs

Some design decisions in the final version were made deliberately to keep the system reliable and practical for the available hardware.

For example:

- Wi-Fi profile selection was implemented through predefined profiles instead of full text entry
- ThingSpeak was chosen as the final cloud backend to align the implementation with the project specification
- the local dashboard was adapted to read from ThingSpeak rather than acting as an isolated parallel backend

These choices improved overall robustness and helped keep the final system consistent.

19. Conclusion

The Greenhouse CO2 Controller project successfully implements a multitask embedded control system with both local and remote functionality. The final system measures greenhouse-related values, controls fan and CO2 valve behavior, stores persistent settings, supports multiple Wi-Fi profiles, communicates with the cloud securely, and allows remote setpoint control.

The project combines several important embedded systems concepts in one application:

- FreeRTOS multitasking
- shared-state synchronization

- peripheral communication through UART, I2C, and GPIO
- persistent EEPROM storage
- local user interface design
- secure cloud integration through HTTPS/TLS
- browser-based monitoring and control

The final implementation goes beyond the minimum required local user interface and persistent setpoint handling by also providing:

- cloud telemetry through ThingSpeak
- remote command handling through TalkBack
- browser-based live dashboard support
- local Wi-Fi profile selection with persistent storage and live switching

Overall, the project demonstrates a complete greenhouse controller prototype that is both technically functional and suitable for demonstration. It provides a practical embedded solution with clear architecture, user interaction, persistent configuration, and cloud connectivity.